```
$numbers = range(2, 5);              // $numbers = array(2, 3, 4, 5);
$letters = range('a', 'z');          // $letters holds the alphabet
$reversedNumbers = range(5, 2);      // $reversedNumbers = array(5, 4, 3, 2);
```

Only the first letter of a string argument is used to build the range:

```
range("aaa", "zzz");                 // same as range('a','z')
```

Getting the Size of an Array

The count() and sizeof() functions are identical in use and effect. They return the number of elements in the array. There is no stylistic preference about which function you use. Here's an example:

```
$family = array("Fred", "Wilma", "Pebbles");
$size = count($family);              // $size is 3
```

This function counts only array values that are actually set:

```
$confusion = array( 10 => "ten", 11 => "eleven", 12 => "twelve");
$size = count($confusion);      // $size is 3
```

Padding an Array

To create an array with values initialized to the same content, use array_pad(). The first argument to array_pad() is the array, the second argument is the minimum number of elements you want the array to have, and the third argument is the value to give any elements that are created. The array_pad() function returns a new padded array, leaving its argument (source) array alone.

Here's array_pad() in action:

```
$scores = array(5, 10);
$padded = array_pad($scores, 5, 0);    // $padded is now array(5, 10, 0, 0, 0)
```

Notice how the new values are appended to the end of the array. If you want the new values added to the start of the array, use a negative second argument:

```
$padded = array_pad($scores, -5, 0);   // $padded is now array(0, 0, 0, 5, 10);
```

If you pad an associative array, existing keys will be preserved. New elements will have numeric keys starting at 0.

5.2.5 Multidimensional Arrays

A multi-dimensional array each element in the main array can also be an array. And each element in the sub-array can be an array, and so on. Values in the multi-dimensional array are accessed using multiple index.